

Pathway / GO Enrichment Analysis

Instructions and Reasoning for pathway_enrichment_reusable.R

What this script does, in one sentence

- Step 1 — Install and load packages
 - Step 2 — Why this is written as a function
 - Step 3 — Load and validate the DEG table
 - Step 4 — Define the “universe” (background gene set)
 - Step 5 — Pull out the significant gene lists
 - Step 6 — Sanity check: do the gene IDs actually match the database?
 - Step 7 — Run GO enrichment
 - Step 8 — Save results (CSV tables and dotplot images)
 - Step 9 — Running it on your actual comparisons
 - Step 10 — How to read the output CSV
- What to do with these results next

What this script does, in one sentence

It takes a list of genes that changed significantly due to radiation (from your DESeq2 output) and asks: **“do these genes cluster into specific biological processes more than you’d expect by random chance?”** That’s a different question from “is this gene significant,” which DESeq2 already answered — enrichment is about the *group* of genes as a whole, not any single one.

This script must be run **after** multifactor_analysis.R, since it reads the DEG_*.csv files that script produces. It doesn’t generate any new expression data — it interprets results you already have.

Step 1 — Install and load packages

Reasoning: clusterProfiler is the engine that actually runs the enrichment statistics. org.At.tair.db is the Arabidopsis-specific annotation database — it’s the lookup table that knows which GO (“Gene Ontology”) biological process each AT1G01010-style gene ID belongs to. Without this package, R has no way to know what any of your genes *do*. enrichplot and ggplot2 are only needed for the dotplot visualizations at the end.

```
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")

pkgs <- c("clusterProfiler", "org.At.tair.db", "enrichplot",
"ggplot2")
for (p in pkgs) {
  if (!requireNamespace(p, quietly = TRUE)) {
    BiocManager::install(p, update = FALSE, ask = FALSE)
  }
}
```

```
library(clusterProfiler)
library(org.At.tair.db)
library(enrichplot)
library(ggplot2)
```

Note: these packages are large and the first install can take several minutes. You only need to install them once — `library()` is the line that needs to run every session.

Step 2 — Why this is written as a function

Reasoning: Rather than copy-pasting the same fifteen lines of code for every comparison (5a, 5b, 5c, 5d...), the whole analysis is wrapped in one function, `run_pathway_enrichment()`. This means:

- Every comparison gets the exact same logic applied consistently — no risk of accidentally using a different threshold or forgetting a step on one comparison but not another.
- Output filenames are generated automatically from a short label you provide, so results never silently overwrite each other.
- If you find a bug or want to change a threshold later, you fix it in one place instead of four.

```
run_pathway_enrichment <- function(deg_csv_path,
                                   label,
                                   padj_cutoff = 0.05,
                                   lfc_cutoff = 0) {
```

The four arguments:

Argument	What it controls
<code>deg_csv_path</code>	Which <code>DEG_*.csv</code> file to read (e.g. from comparison 5a, 5b, etc.)
<code>label</code>	A short name used to build output filenames, e.g. "OSD498_510_radiation"
<code>padj_cutoff</code>	The significance threshold for calling a gene "differentially expressed" (default: 0.05)
<code>lfc_cutoff</code>	An optional minimum fold-change size to also require (default: 0, meaning off)

Step 3 — Load and validate the DEG table

Reasoning: Before doing anything else, the function checks that the file you pointed it at actually looks like a DESeq2 results table (it must have `log2FoldChange` and `padj` columns). This isn't excessive caution — if you accidentally pass it the wrong file, you want a clear error message immediately, not a confusing failure three steps later inside the enrichment statistics themselves.

```
res_df <- read.csv(deg_csv_path, row.names = 1)
```

```

required_cols <- c("log2FoldChange", "padj")
missing_cols <- setdiff(required_cols, colnames(res_df))
if (length(missing_cols) > 0) {
  stop("Input file is missing expected column(s): ",
paste(missing_cols, collapse = ", "),
      " -- is this really a DESeq2 results CSV?")
}

```

Step 4 — Define the “universe” (background gene set)

Reasoning — this is the single most important and most commonly mis-done step in pathway enrichment. The “universe” is the background your significant genes get compared against. It must be **every gene that DESeq2 actually tested** in that comparison — not the entire Arabidopsis genome, and not some other arbitrary list. Using the wrong universe silently changes the meaning of “more than expected by chance,” and can make weak results look artificially strong (or hide real ones). Since `rownames(res_df)` is exactly the set of genes DESeq2 kept after its own internal filtering, that’s the correct, honest universe to use here.

```

universe <- rownames(res_df)
cat("Universe size (genes tested):", length(universe), "\n")

```

Step 5 — Pull out the significant gene lists

Reasoning: This is the same up/down logic from your earlier DEG review, just generalized with adjustable thresholds. A gene counts as **upregulated** if it passed the significance cutoff and has a *positive* fold-change; **downregulated** if significant with a *negative* fold-change. The optional `lfc_cutoff` lets you also require a minimum fold-change magnitude (e.g. at least 2-fold) if you want to filter out changes that are statistically real but biologically tiny — by default this is turned off (0), so only statistical significance is required.

```

sig_up <- rownames(subset(res_df,
                          padj < padj_cutoff &
                          log2FoldChange > lfc_cutoff))
sig_down <- rownames(subset(res_df,
                             padj < padj_cutoff &
                             log2FoldChange < -lfc_cutoff))

```

If neither list has any genes, the function stops early and returns empty results rather than feeding zero genes into the enrichment step, which would just error out anyway.

```

if (length(sig_up) == 0 && length(sig_down) == 0) {
  cat("No significant genes found at this threshold -- skipping
enrichment.\n")
  return(invisible(list(label = label, sig_up = sig_up, sig_down =
sig_down,
                        ego_up = NULL, ego_down = NULL)))
}

```

Step 6 — Sanity check: do the gene IDs actually match the database?

Reasoning: This step exists because of a very common, very confusing failure mode: enrichment analysis can run *without any error* and still return zero results — not because there’s no real biology, but because the gene ID format in your file doesn’t match what the annotation database expects. Catching this early, with a clear warning, saves you from puzzling over an empty results table later wondering whether the biology itself is the problem.

```
check_ids <- head(c(sig_up, sig_down), 20)
if (length(check_ids) > 0) {
  mapped_check <- suppressWarnings(
    AnnotationDbi::select(org.At.tair.db, keys = check_ids,
                          columns = "GO", keytype = "TAIR")
  )
  n_mapped <- length(unique(mapped_check$TAIR))
  cat("ID mapping sanity check:", n_mapped, "of", length(check_ids),
      "sampled genes mapped to at least one GO term\n")
  if (n_mapped == 0) {
    warning("ZERO genes mapped to the annotation database. Check
that your ",
            "gene IDs are in TAIR locus format (e.g. AT1G01010) --
enrichment ",
            "results below will likely be empty.")
  }
}
```

Since this project’s gene IDs are already confirmed to be in AT1G01010-style TAIR format, this check is expected to pass — but it’s left in so the function stays safe to reuse on different data later.

Step 7 — Run GO enrichment

Reasoning for ont = "BP": GO terms come in three categories — Biological Process (what the gene’s product *does*, e.g. “DNA repair”), Molecular Function (its specific biochemical activity), and Cellular Component (where in the cell it’s located). Biological Process is usually the most directly interpretable for a question like “how did the plant respond to radiation,” so that’s the default here. (ont = "ALL" would return all three categories together if you want a broader picture later.)

Reasoning for running up and down separately: Combining up- and down-regulated genes into one enrichment test can hide real patterns — for instance, if DNA repair genes go up while photosynthesis genes go down, testing them together would blur both signals. Splitting them keeps each result interpretable as a clear direction of change.

```
ego_up <- if (length(sig_up) > 0) {
  enrichGO(gene = sig_up, universe = universe, OrgDb =
org.At.tair.db,
           keyType = "TAIR", ont = "BP", pAdjustMethod = "BH",
           pvalueCutoff = 0.05, qvalueCutoff = 0.2)
} else NULL
```

```
ego_down <- if (length(sig_down) > 0) {
  enrichGO(gene = sig_down, universe = universe, OrgDb =
org.At.tair.db,
           keyType = "TAIR", ont = "BP", pAdjustMethod = "BH",
           pvalueCutoff = 0.05, qvalueCutoff = 0.2)
} else NULL
```

pAdjustMethod = "BH" (Benjamini-Hochberg) is the same multiple-testing correction logic as DESeq2's padj column — necessary here too, since you're testing hundreds of GO terms simultaneously, and without correction many would appear "significant" purely by chance.

Step 8 — Save results (CSV tables and dotplot images)

Reasoning: Each direction (up/down) gets its own CSV and its own dotplot, named using the label you provided — this is what makes the function safe to call repeatedly across 5a/5b/5c/5d without overwriting anything. The `if (!is.null(...) && nrow(...) > 0)` checks mean the script won't crash or produce a broken empty plot if a particular comparison genuinely has zero enriched terms — it just reports that cleanly instead.

```
if (!is.null(ego_up) && nrow(as.data.frame(ego_up)) > 0) {
  out_csv <- paste0("GO_enrichment_", label, "_upregulated.csv")
  write.csv(as.data.frame(ego_up), out_csv, row.names = FALSE)
  cat("Saved:", out_csv, "(", nrow(as.data.frame(ego_up)), "enriched
terms )\n")

  p_up <- dotplot(ego_up, showCategory = 15) +
    ggtitle(paste0(label, " -- UPregulated"))
  ggsave(paste0("GO_dotplot_", label, "_upregulated.png"), p_up,
width = 9, height = 7)
} else {
  cat("No enriched terms found for upregulated genes (or no genes to
test).\n")
}
```

(The downregulated block mirrors this exactly.)

Step 9 — Running it on your actual comparisons

Reasoning: Each call below is independent — you can run them in any order, and you only need the relevant DEG_*.csv file to exist before calling it. The commented-out lines are pre-written for 5b/5c/5d so you just uncomment and run once those files are available, without needing to re-derive the correct filenames or labels yourself.

```
results_5a <- run_pathway_enrichment(
  deg_csv_path = "DEG_OSD498_510_radiation_effect.csv",
  label       = "OSD498_510_radiation"
)
```

```

# results_5b <- run_pathway_enrichment(
#   deg_csv_path =
"DEG_OSD508_510_genotype_x_radiation_interaction.csv",
#   label       = "OSD508_510_interaction"
# )

# results_5c_low <- run_pathway_enrichment(
#   deg_csv_path = "DEG_OSD782_10cGy_vs_none.csv",
#   label       = "OSD782_10cGy"
# )

# results_5c_high <- run_pathway_enrichment(
#   deg_csv_path = "DEG_OSD782_100cGy_vs_none.csv",
#   label       = "OSD782_100cGy"
# )

# results_5d_40 <- run_pathway_enrichment(
#   deg_csv_path = "DEG_OSD658_GCR40_vs_none.csv",
#   label       = "OSD658_GCR40"
# )

# results_5d_80 <- run_pathway_enrichment(
#   deg_csv_path = "DEG_OSD658_GCR80_vs_none.csv",
#   label       = "OSD658_GCR80"
# )

```

Important caveat for 5b specifically: the interaction term’s log2FoldChange doesn’t mean simple “up” or “down” expression — it means “the size of the radiation effect differs between sog1_1 and WT by this much.” Running this function on the interaction file will still work mechanically, but when interpreting the resulting GO terms, phrase them as “*processes where the radiation response itself differs by genotype*,” not “*processes upregulated by radiation*.”

Step 10 — How to read the output CSV

Each saved GO_enrichment_*.csv has these columns:

Column	What it means
Description	The biological process name, e.g. “DNA repair” or “response to gamma radiation”
GeneRatio	(your significant genes in this term) / (total significant genes you tested)
BgRatio	(genes in this term overall) / (total genes in the universe)
pvalue / p.adjust / qvalue	Statistical significance — filter on p.adjust , the same logic as DESeq2’s padj
geneID	Which of <i>your</i> genes are driving this particular enrichment
Count	How many of your genes fall into this term

A result is genuinely meaningful enrichment — not just a large, generic category — when GeneRatio is noticeably higher than BgRatio **and** $p.adjust < 0.05$. A term with a tiny $p.adjust$ but only 2 genes behind it is worth treating cautiously; a term with both strong statistics and a reasonable gene count is the most trustworthy kind of result to report or follow up on.

What to do with these results next

1. Sort each CSV by $p.adjust$ and look at the top 10-15 rows — these are your headline findings for that comparison.
2. Open the matching `GO_dotplot_*.png` for a visual summary — dot size shows how many genes are involved, color shows significance.
3. Cross-check whether the same biological processes show up across multiple comparisons (e.g. does “DNA repair” appear in both 5a and 5b’s results?) — consistency across independent comparisons is one of the strongest forms of evidence in this kind of multi-study analysis.
4. For any term you want to discuss in a write-up, look at the `geneID` column to identify the specific genes responsible — that’s often more convincing than the GO term name alone.